

Практическая работа №4

Непрерывная интеграция (CI) с Docker и GitHub Actions

✂ Цель работы:

Освоить процесс автоматизированной сборки и тестирования Docker-образа с помощью GitHub Actions.

✂ Задачи:

- 1. Настроить GitHub Actions для автоматической сборки Docker-образа.
- 2. Проверить код с помощью unit-тестов перед сборкой.
- 3. Загружать собранный образ в Docker Hub.
- 4. (Дополнительно) Развернуть контейнер на сервере.

1. Подготовка проекта

◇ Создание локального репозитория:

```
$ mkdir ci_docker_project  
$ cd ci_docker_project  
$ git init
```

◇ Связывание с удалённым репозиторием на GitHub:

```
$ git remote add origin https://github.com/your-username/ci_docker_project.git
```

2. Написание тестового веб-приложения

◇ Создание файлов проекта:

```
$ touch app.py  
$ touch requirements.txt
```

◇ Код `app.py`:

```
from flask import Flask  
app = Flask(__name__)  
@app.route('/')  
def home():  
    return 'Hello, Docker CI/CD!'  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```

◇ Файл зависимостей `requirements.txt`:

flask

3. Создание тестов

- ◇ Создаём тесты в `test\_app.py`:

```
$ mkdir tests
```

```
$ touch tests/test_app.py
```

- ◇ Код `tests/test\_app.py`:

```
import pytest
```

```
from app import app
```

```
@pytest.fixture
```

```
def client():
```

```
    with app.test_client() as client:
```

```
        yield client
```

```
def test_home(client):
```

```
    response = client.get('/')
```

```
    assert response.status_code == 200
```

```
    assert response.data == b'Hello, Docker CI/CD!'
```

4. Создание Dockerfile

- ◇ Создаём `Dockerfile`:

```
FROM python:3.9-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt requirements.txt
```

```
COPY app.py app.py
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
EXPOSE 5000
```

```
CMD ['python', 'app.py']
```

5. Настройка GitHub Actions

- ◇ Создаём workflow для автоматического тестирования и сборки:

```
$ mkdir -p .github/workflows
```

```
$ touch .github/workflows/ci.yml
```

- ◇ Код `.github/workflows/ci.yml`:

```
name: CI/CD Pipeline
```

```
on:
```

push:
 branches:
 - main
pull_request:

jobs:
 test:
 runs-on: ubuntu-latest
 steps:
 - name: Checkout repository
 uses: actions/checkout@v3

 - name: Set up Python
 uses: actions/setup-python@v4
 with:
 python-version: '3.9'

 - name: Install dependencies
 run: |
 pip install -r requirements.txt
 pip install pytest

 - name: Run tests
 run: pytest

build:
 needs: test
 runs-on: ubuntu-latest
 steps:
 - name: Checkout repository
 uses: actions/checkout@v3

 - name: Build Docker image
 run: docker build -t myapp:latest .

6. Проверка работы CI/CD

◇ Коммитим и пушим код:

```
$ git add .  
$ git commit -m 'Добавлен CI/CD pipeline'  
$ git push origin main
```

◇ Проверяем GitHub Actions:

1. Перейдите в репозиторий → Actions.
2. Убедитесь, что тесты и сборка Docker проходят успешно.

7. Развертывание контейнера на сервере (Дополнительно)

- ◇ Подключаемся к серверу через SSH:

```
$ ssh user@your-server-ip
```

- ◇ Загружаем образ с Docker Hub:

```
$ docker pull yourdockerhub/myapp:latest
```

- ◇ Запускаем контейнер:

```
$ docker run -d -p 80:5000 yourdockerhub/myapp:latest
```

- ◇ Проверяем в браузере:

```
http://your-server-ip
```

8. Ожидаемый результат

- ☒ Настроен GitHub Actions Workflow для тестирования и сборки Docker-образа
- ☒ Тесты выполняются при каждом коммите
- ☒ Образ автоматически отправляется в Docker Hub
- ☒ (Дополнительно) Контейнер развёрнут на сервере

9. Сам работа

- ◇ Добавить тестирование на разных версиях Python.
- ◇ Использовать `docker-compose` для автоматизации запуска контейнера.
- ◇ Настроить автоматический деплой в Kubernetes.

10. Формат сдачи работы

1. Ссылка на GitHub-репозиторий с `Dockerfile` и `.github/workflows/ci.yml`.
2. Скриншоты успешного выполнения GitHub Actions Workflow.
3. (Дополнительно) Ссылка на Docker Hub с опубликованным образом.
4. (Дополнительно) IP-адрес сервера с запущенным контейнером.